Final Assignment of 1 Mark

Name: Soniya Ekanth Girhe.

Sec_batch_RollNo.: A7_B3_59.

1. (i) Generate a user defined formulae to find the surface area and volume of a cylinder. (ii) Solve the system of equation x 2 − y 2 = 5, x + y = 5

In [3]:
```
def cylinder(r, h):
    SA = 2 * pi * r * (r + h)
    V = pi * r^2 * h
    print("Surface Area =", SA)
    print("Volume =", V)

cylinder(3, 5)  # Example: radius=3, height=5
```

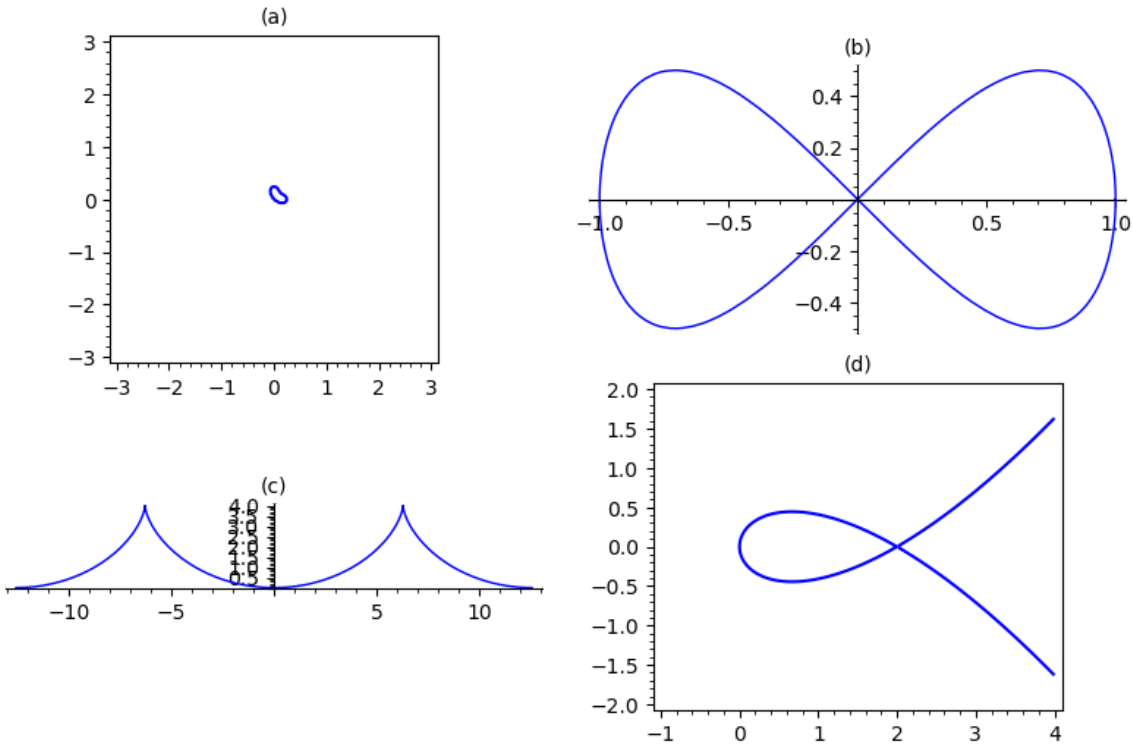
```
Surface Area = 48*pi
Volume = 45*pi
```

In [4]:
```
x, y = var('x y')
sol = solve([x^2 - y^2 == 5, x + y == 5], x, y)
print(sol)
```

```
[
[x == 3, y == 2]
]
```

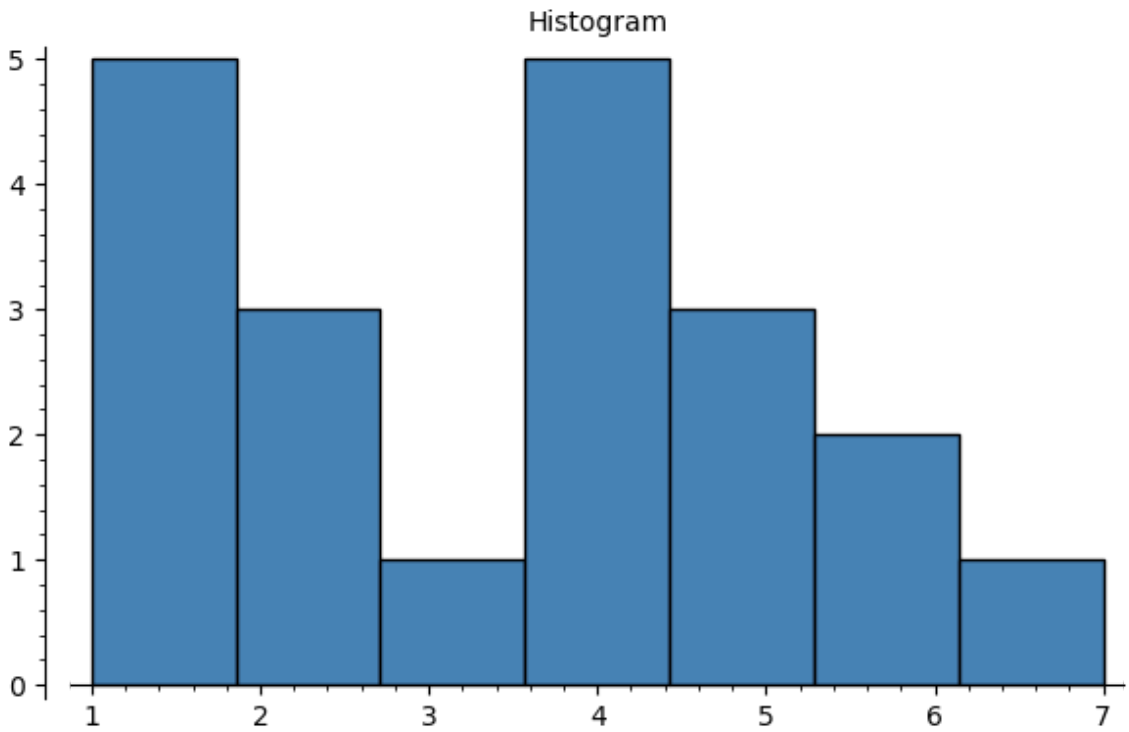
1. (i) Plot and arrange the graphs in a 2 by 2 grid: (a)(x 3 + y 3) − 4(x 2 + y 2) 2 = 0, (b) x(t) = cos t, y(t) = cos t sin t, (c) x(t) = 2(t + sin t), y(t) = 2(1 − cos t) (d) 6y2 = x(x − 2)

In [5]:
```
t = var('t')

p1 = implicit_plot(x^3 + y^3 - 4*(x^2 + y^2)^2, (x,-3,3), (y,-3,3), title
='(a)')
p2 = parametric_plot((cos(t), cos(t)*sin(t)), (t, 0, 2*pi), title='(b)')
p3 = parametric_plot((2*(t+sin(t)), 2*(1-cos(t))), (t, -2*pi, 2*pi), title
='(c)')
p4 = implicit_plot(6*y^2 - x*(x-2)^2, (x,-1,4), (y,-2,2), title='(d)')

G = graphics_array([[p1, p2], [p3, p4]])
G.show(figsize=8)
```



(ii) Draw a histogram for the dataset 1,2,4,1,1,5,7,4,5,6,5,6,1,2,3,4,4,4,1,2

In [6]:
```
data = [1,2,4,1,1,5,7,4,5,6,5,6,1,2,3,4,4,4,1,2]
histogram(data, bins=7, edgecolor='black', color='steelblue').show(title='H
istogram')
```



1. (a) Check whether the vectors (2,3,5,7), (1,2,3,4), (1,1,2,3) are linearly independent or dependent.

In [7]:
```
M = matrix([[2,3,5,7],[1,2,3,4],[1,1,2,3]])
if M.rank() == 3:
    print("The vectors are Linearly INDEPENDENT")
else:
    print("The vectors are Linearly DEPENDENT")
print("Rank =", M.rank())
```

```
The vectors are Linearly DEPENDENT
Rank = 2
```

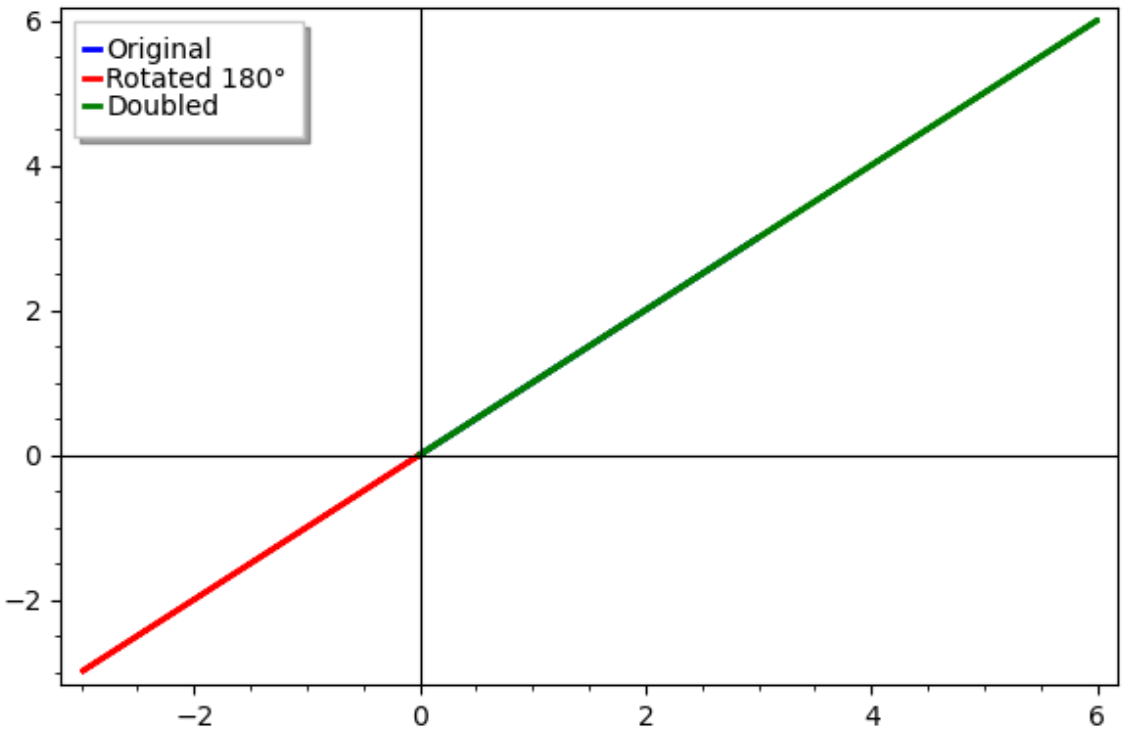
(b) Generate a 4th order square matrix whose all entries are 2. Replace the diagonals by 1,3,5,7 respectively. Then Replace the first row by 3 and 3rd column by 5. Hence find the Eigen values.

In [8]:
```
A = matrix(RR, 4, 4, [2]*16)

# Replace diagonals by 1,3,5,7
for i, val in enumerate([1,3,5,7]):
    A[i,i] = val

# Replace first row by 3
A[0] = [3,3,3,3]

# Replace 3rd column (index 2) by 5
for i in range(4):
    A[i,2] = 5

print("Matrix A:")
print(A)
print("\nEigenvalues:", A.eigenvalues())
```

```
Matrix A:
[3.00000000000000 3.00000000000000 5.00000000000000 3.00000000000000]
[2.00000000000000 3.00000000000000 5.00000000000000 2.00000000000000]
[2.00000000000000 2.00000000000000 5.00000000000000 2.00000000000000]
[2.00000000000000 2.00000000000000 5.00000000000000 7.00000000000000]

/opt/sagemath-9.3/local/lib/python3.7/site-packages/sage/repl/ipython_kern
el/__main__.py:16: UserWarning: Using generic algorithm for an inexact rin
g, which will probably give incorrect results due to numerical precision i
ssues.

Eigenvalues: [13.0281142401025, 3.69957236807046, 0.636156695913530 - 0.33
7629266550362*I, 0.636156695913530 + 0.337629266550362*I]
```

1. (i) Join the points(0,0), (1,1), (3,3), (1,1), (0,0). Hence rotate the given figure by 180 degree using Appropriate Matrix Transformation. Also double the size of the original figure.

In [9]:
```
pts = [(0,0),(1,1),(3,3),(1,1),(0,0)]

original = line(pts, color='blue', legend_label='Original', thickness=2)

# Rotation by 180 degrees
R = matrix([[cos(pi), -sin(pi)],[sin(pi), cos(pi)]])
rot_pts = [tuple(R * vector(p)) for p in pts]
rotated = line(rot_pts, color='red', legend_label='Rotated 180°', thickness
=2)

# Double size
doubled_pts = [(2*x, 2*y) for x,y in pts]
doubled = line(doubled_pts, color='green', legend_label='Doubled', thicknes
s=2)

show(original + rotated + doubled, axes=True, frame=True)
```

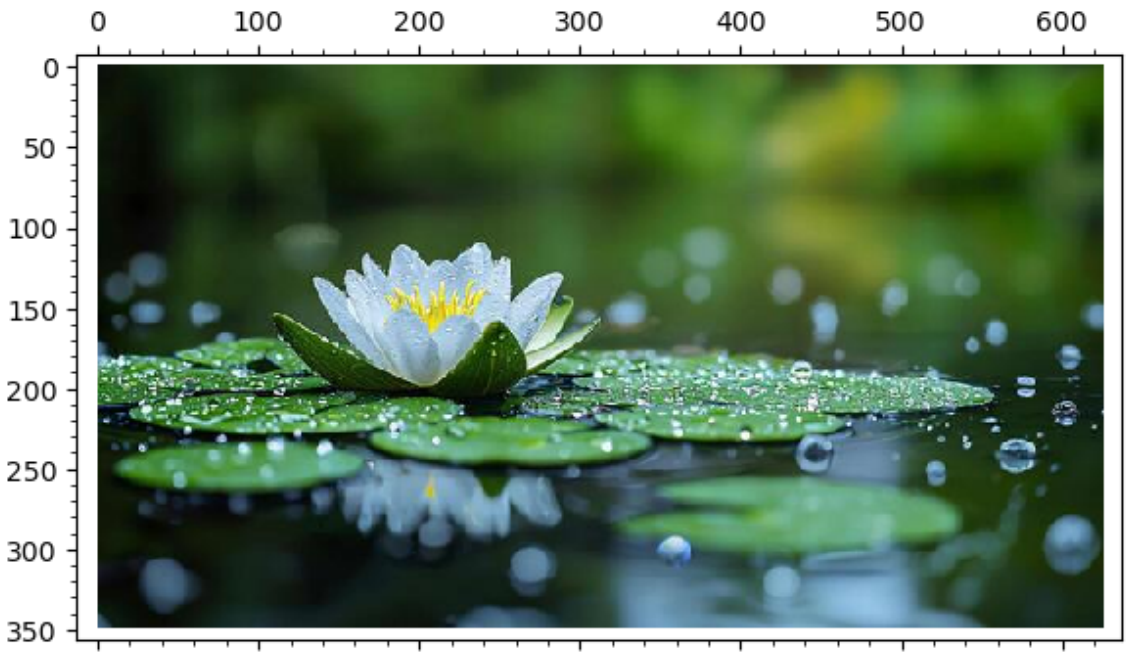


(ii) Using an image of your choice, find its singular value decomposition. Also reduce it dimension and generate the approximated images.

In [24]:
```
from matplotlib.pyplot import imread
import pylab
import numpy as np
img=pylab.imread('img1.png')
```

In [25]:
```
matrix_plot(img)
```

Out[25]:



In [26]:
```
img.shape
```

Out[26]:
```
(350, 625, 3)
```

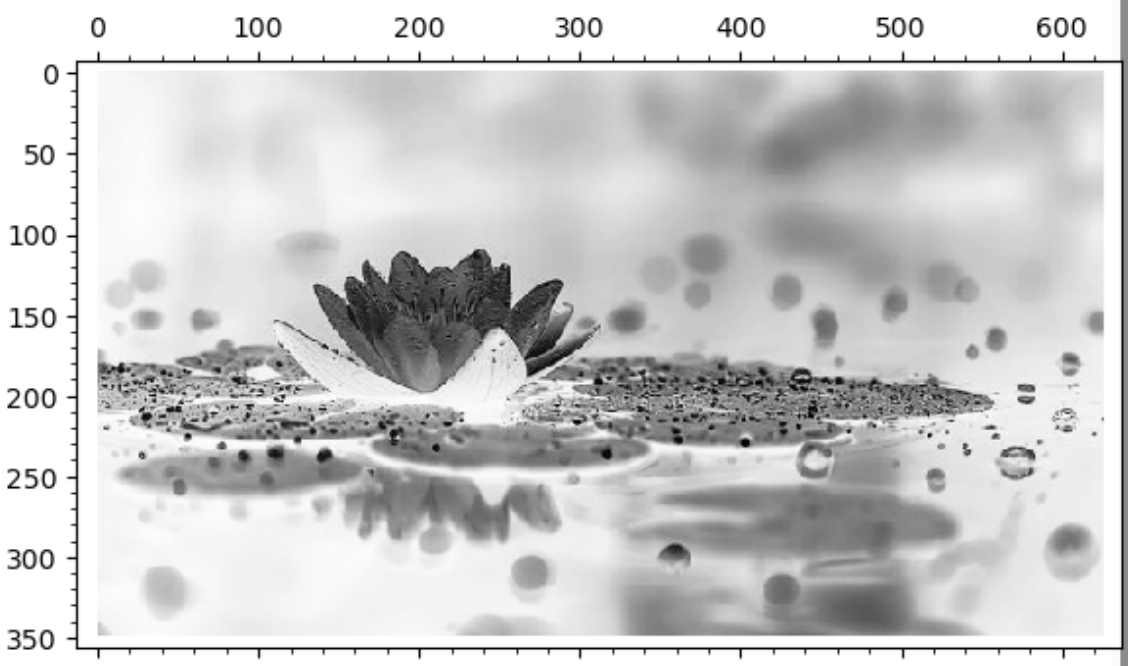
In [27]:
```
show(img)
```

```
[[[0.04313726 0.05882353 0.05490196]
  [0.04313726 0.05882353 0.05490196]
  [0.04313726 0.05882353 0.05490196]
  ...
  [0.15686275 0.2901961  0.05490196]
  [0.15686275 0.2901961  0.05098039]
  [0.15686275 0.2901961  0.05098039]]

 [[0.04313726 0.05882353 0.05490196]
  [0.04313726 0.05882353 0.05490196]
  [0.04313726 0.05882353 0.05490196]
  ...
  [0.16078432 0.29411766 0.05490196]
  [0.16078432 0.29411766 0.05490196]
  [0.16078432 0.29803923 0.04705882]]

 [[0.04313726 0.0627451  0.04705882]
  [0.04313726 0.0627451  0.04705882]
  [0.04313726 0.0627451  0.04705882]
  ...
  [0.16862746 0.30588236 0.05490196]
  [0.16862746 0.30588236 0.04705882]
  [0.16862746 0.30588236 0.04705882]]

 ...

 [[0.22352941 0.35686275 0.4627451 ]
  [0.22352941 0.35686275 0.45490196]
  [0.23137255 0.35686275 0.44705883]
  ...
  [0.0627451  0.13333334 0.08627451]
  [0.0627451  0.13333334 0.08627451]
  [0.0627451  0.13333334 0.07843138]]

 [[0.23137255 0.36862746 0.4862745 ]
  [0.22352941 0.36078432 0.47058824]
  [0.23137255 0.3529412  0.4627451 ]
  ...
  [0.0627451  0.13333334 0.08627451]
  [0.0627451  0.13333334 0.08627451]
  [0.0627451  0.13333334 0.08627451]]

 [[0.23137255 0.37254903 0.49803922]
  [0.22745098 0.3647059  0.48235294]
  [0.21960784 0.3529412  0.4627451 ]
```

```
...
[0.0627451  0.13333334 0.08627451]
[0.05882353 0.12941177 0.08235294]
[0.05882353 0.12941177 0.08235294]]]
```

In [28]:
```
gray=lambda rgb: np.dot(rgb[...,:3],[0.3,0.6,0.1])
G=gray(img)
matrix_plot(G)
```

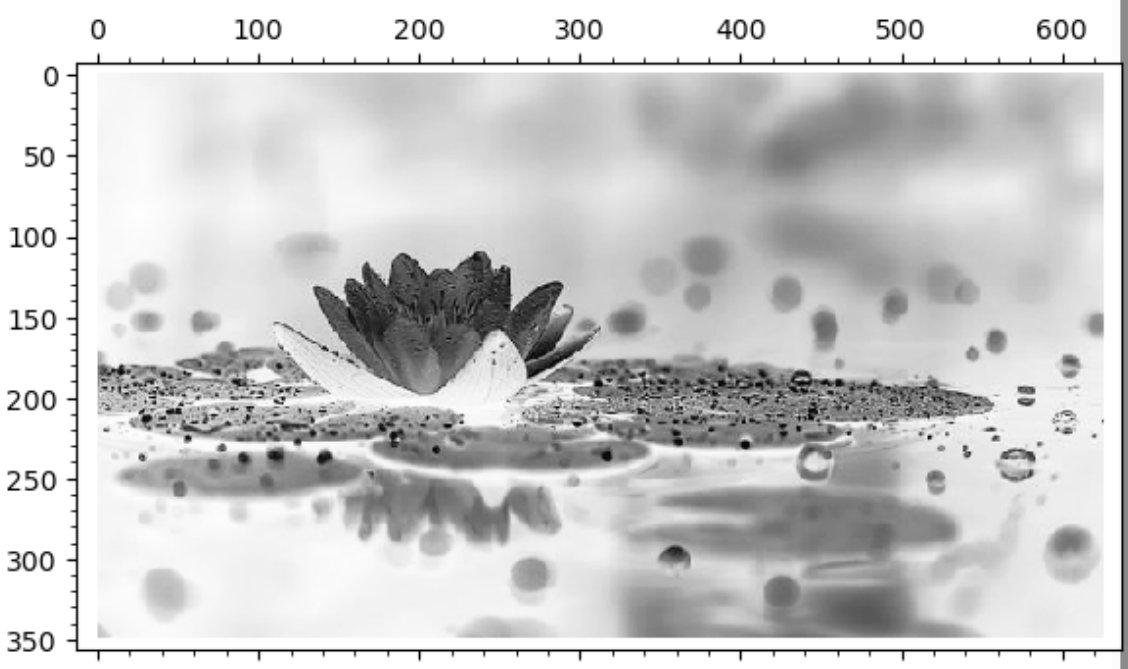
Out[28]:



In [29]:
```
U,S,V=matrix(G).SVD()
```

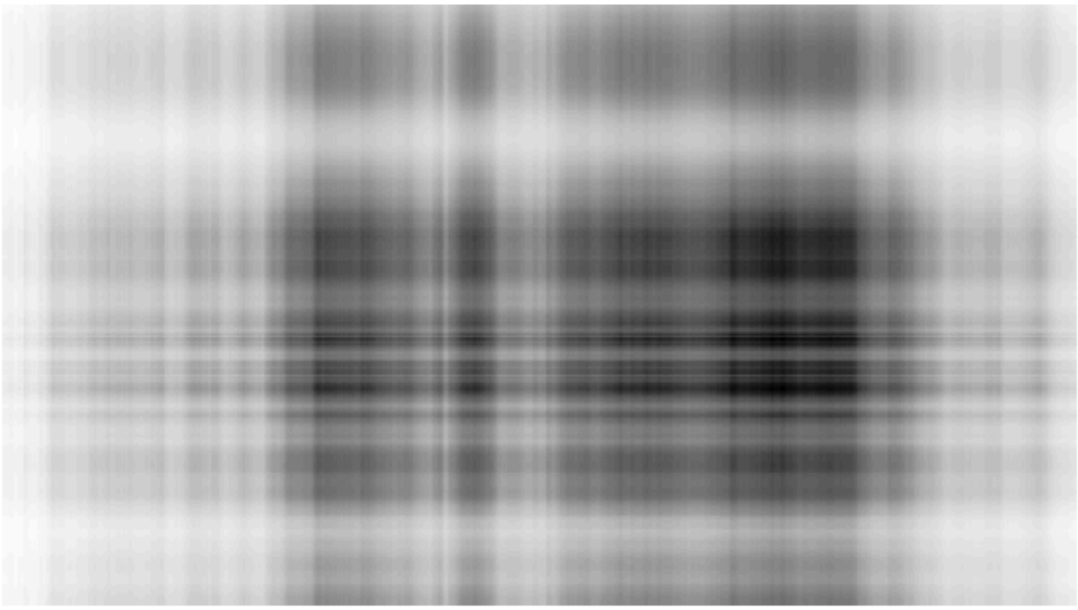
In [30]:
```
n=200
G1=U[:,:n]*S[:n,:n]*V.T[:n,:]
reduced=matrix_plot(G1)
reduced
```

Out[30]:

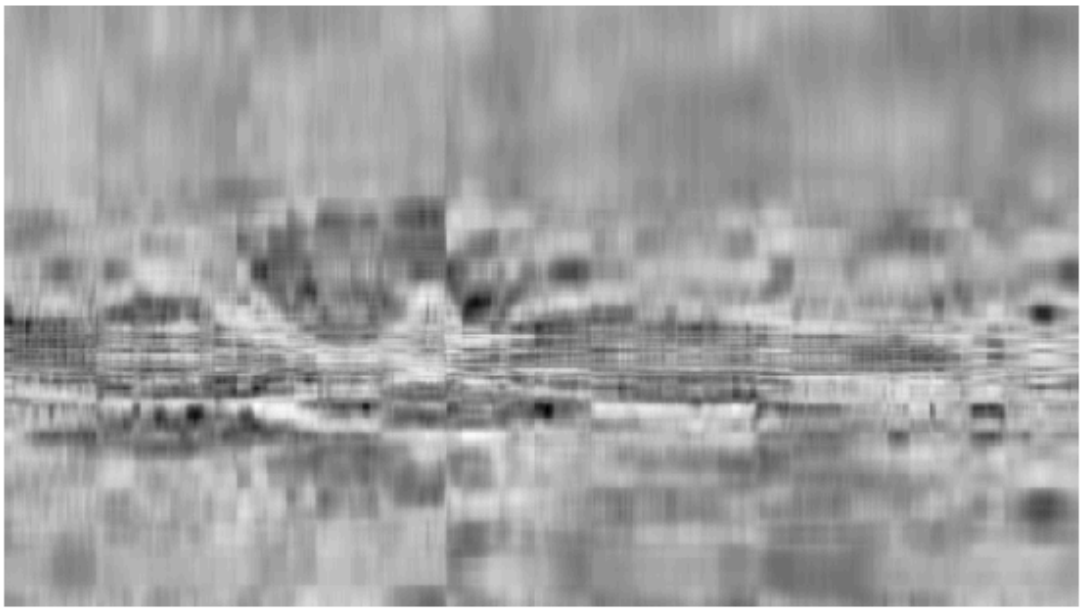


In [31]:
```
appx =[]
for i in range(1,100,10):
    A_approx = U[:,:i]*V.T[:i,:]
    appx_img=matrix_plot(A_approx,title="Using "+str(i)+'Singular Values',
                         frame=False)
    show(appx_img,figsize=6)
```

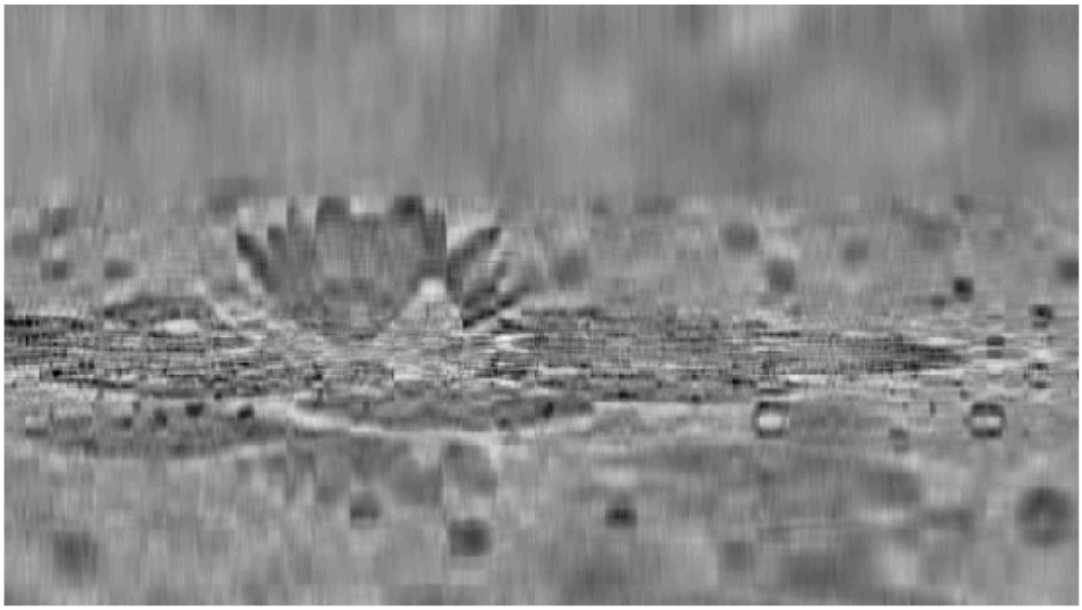
Using 1Singular Values



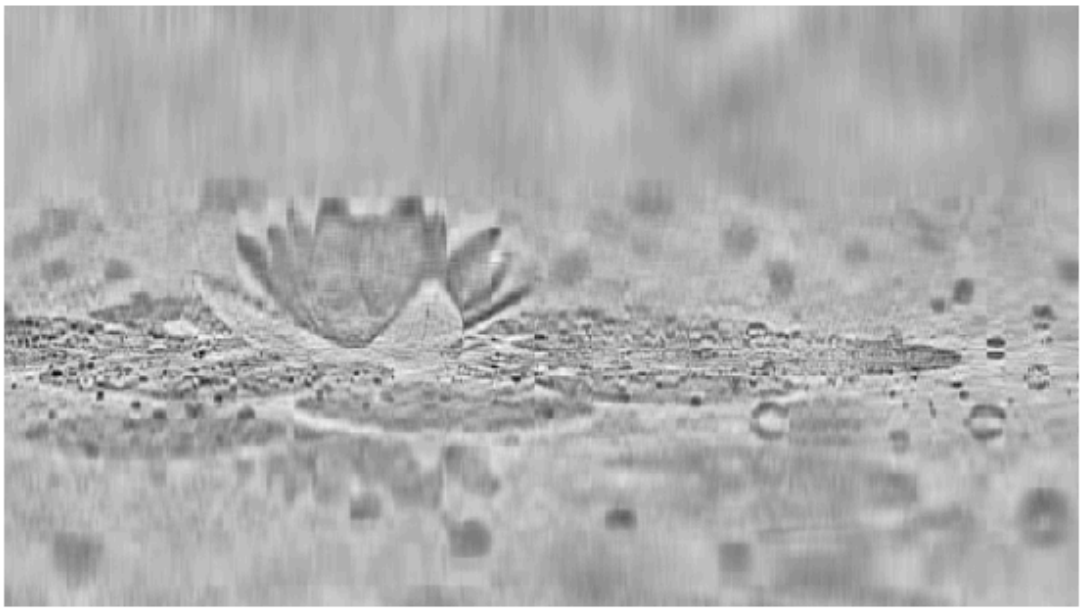
Using 11Singular Values



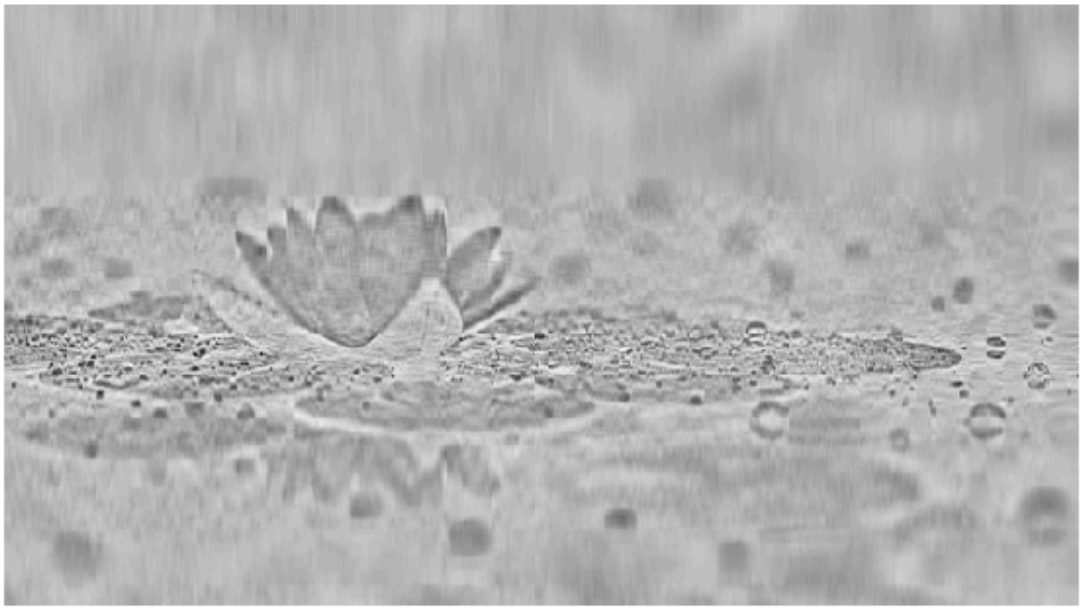
Using 21Singular Values



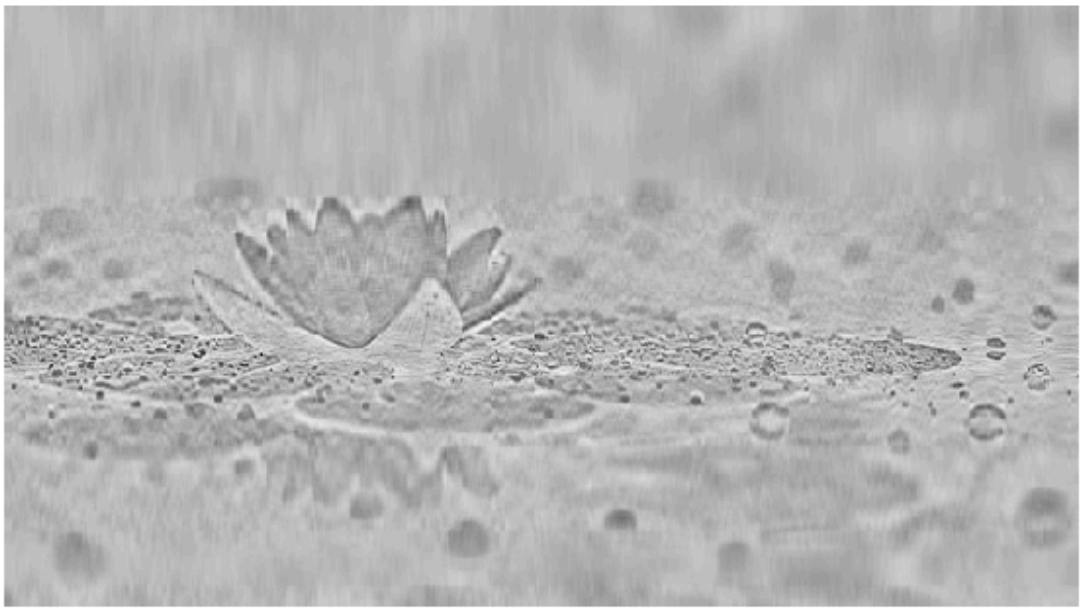
Using 31Singular Values



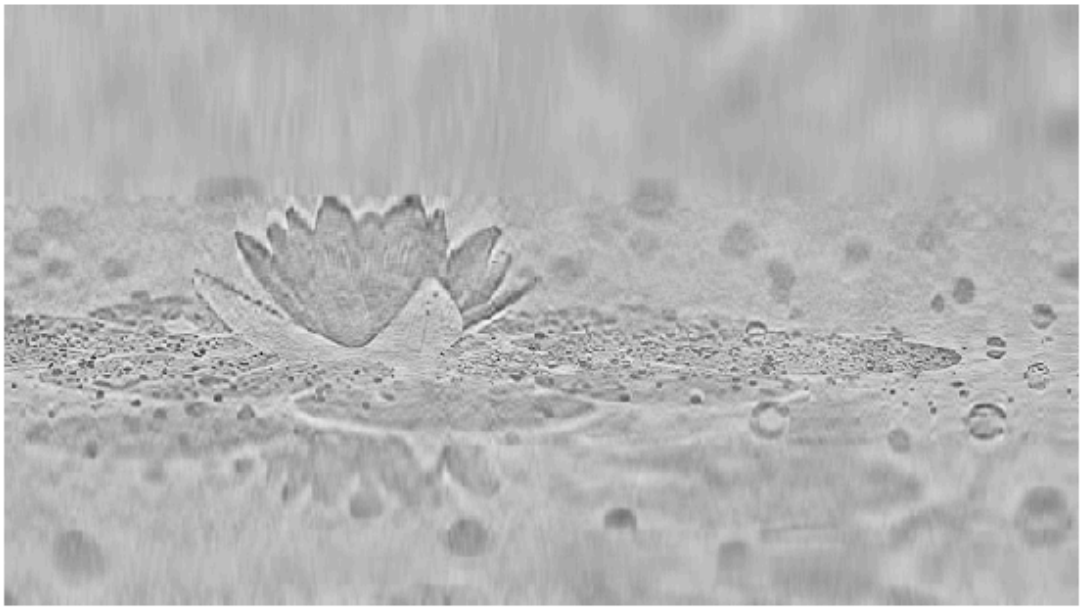
Using 41Singular Values



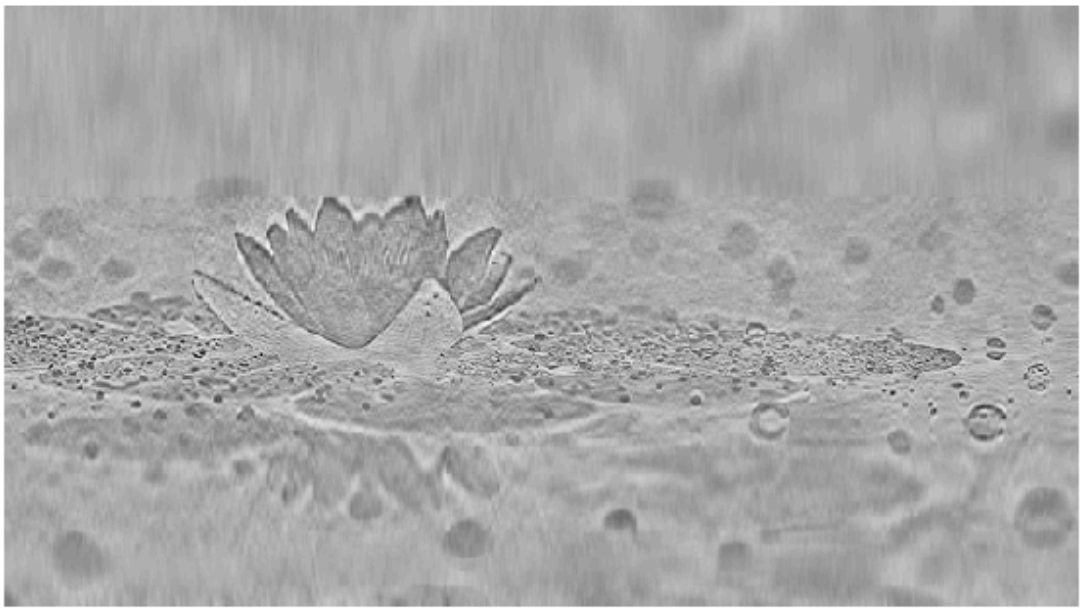
Using 51Singular Values



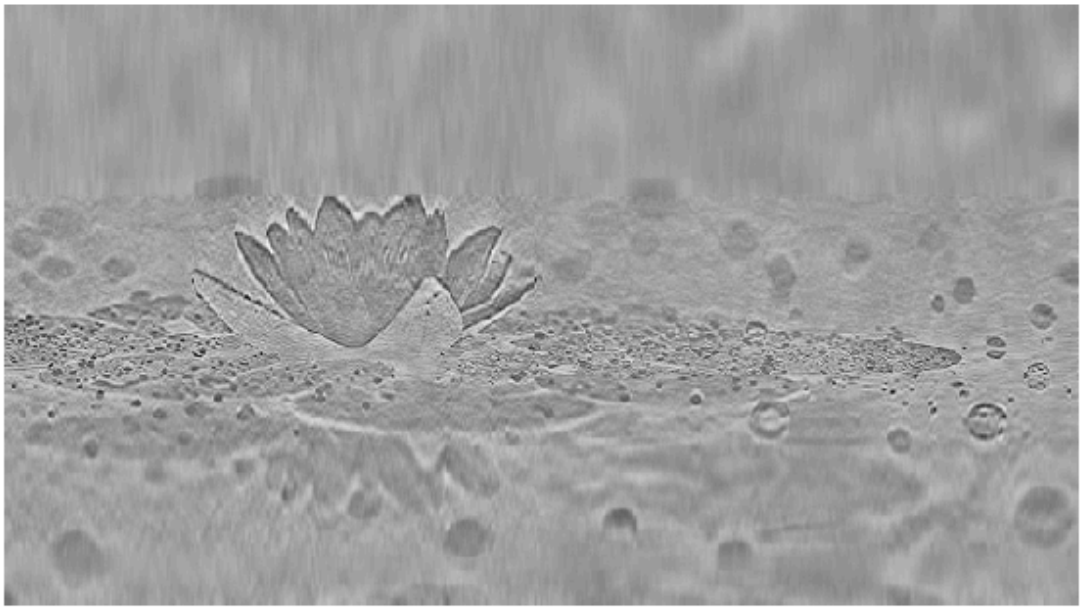
Using 61Singular Values



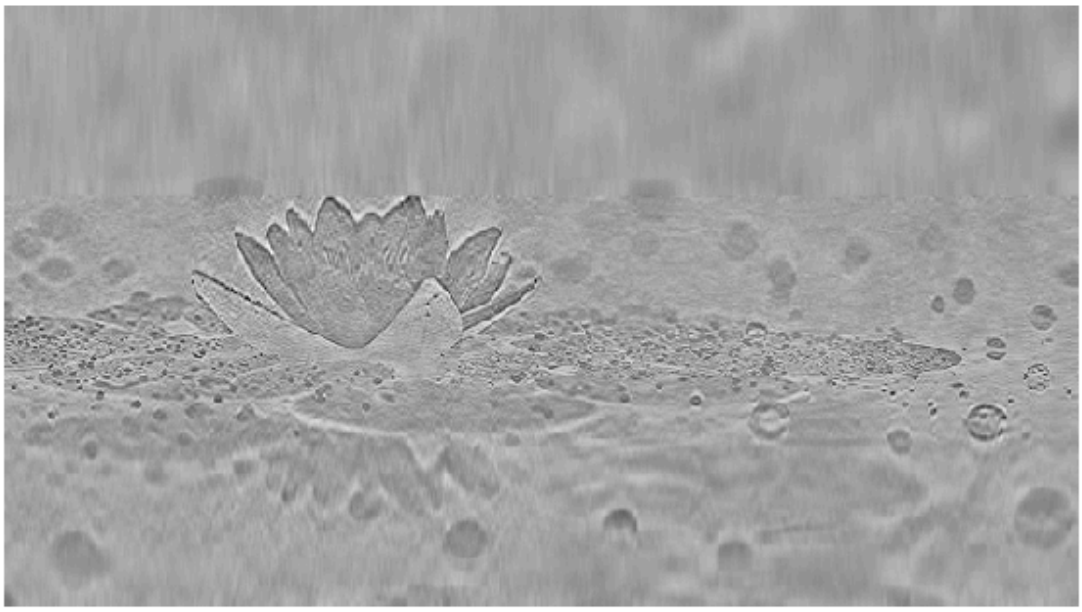
Using 71Singular Values



Using 81Singular Values



Using 91Singular Values

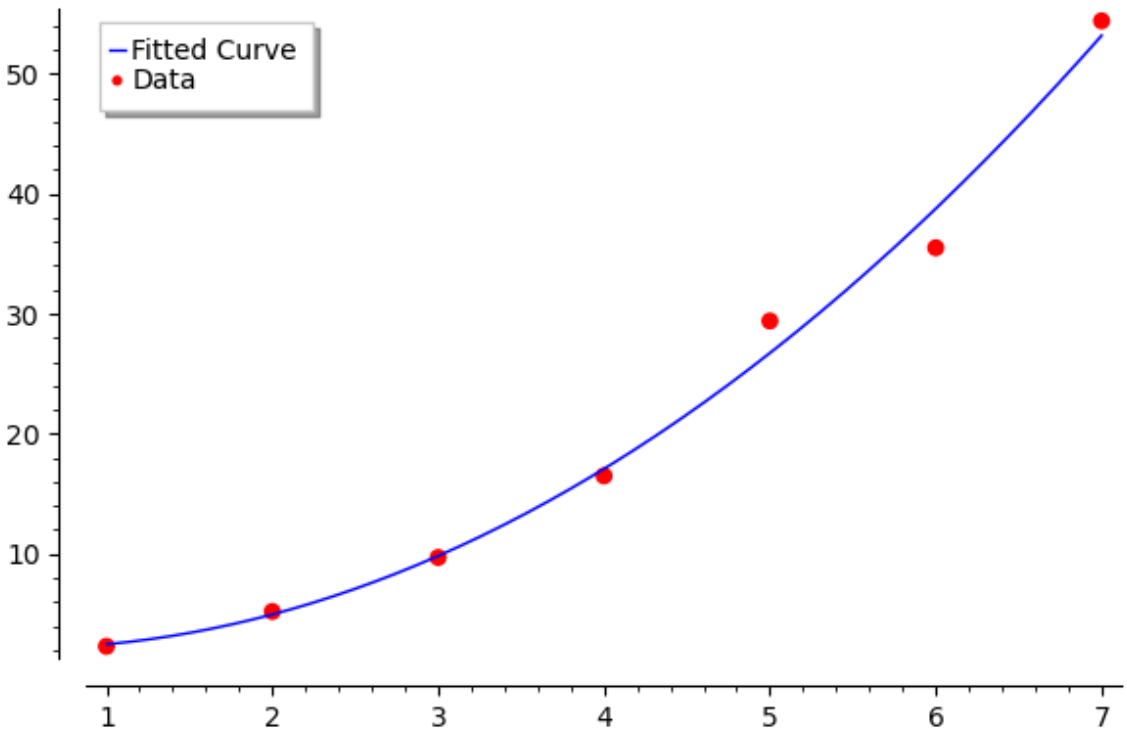


1. Fit a curve y = a + bx + cx 2 to the following data:

x 1 2 3 4 5 6 7 y 2.3 5.2 9.7 16.5 29.4 35.5 54.4

In [32]:
```
var('a b c x')
data = [(1,2.3),(2,5.2),(3,9.7),(4,16.5),(5,29.4),(6,35.5),(7,54.4)]
model = a + b*x + c*x^2

result = find_fit(data, model, parameters=[a,b,c], variables=[x], initial_g
uess=[1,1,1])
print("Fitted parameters:", result)

# Plot fitted curve vs data
pts = list_plot(data, color='red', size=40, legend_label='Data')
curve = plot(result[0].rhs() + result[1].rhs()*x + result[2].rhs()*x^2,
             (x,1,7), color='blue', legend_label='Fitted Curve')
show(pts + curve)
```

```
Fitted parameters: [a == 2.3714285714315757, b == -1.0928571428617144, c =
= 1.1928571428575643]
```



1. Let an be the number of binary strings of length n that do not contain the substring101. How many such numbers are possible?

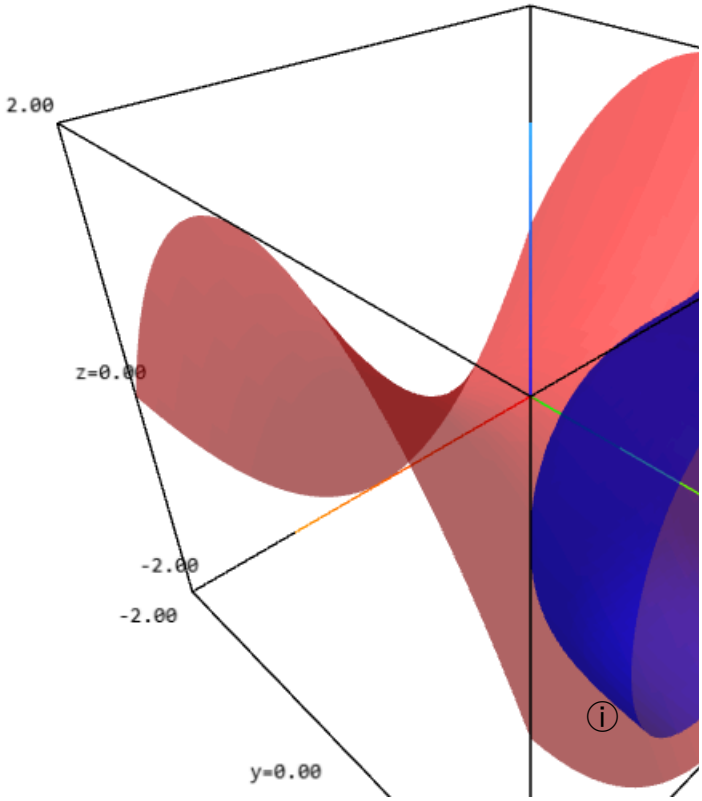
In [33]:
```
# States: s0 = neutral/ends in 0, s1 = ends in 1, s10 = ends in 10
def count_no_101(n):
    s0, s1, s10 = 1, 1, 0  # base case n=1
    for _ in range(n - 1):
        s0, s1, s10 = s0 + s10, s0 + s1, s1
    return s0 + s1 + s10

print(f"{'n':>5} | {'a_n (count)':>12}")
print("-" * 22)
for n in range(1, 11):
    print(f"{n:>5} | {count_no_101(n):>12}")
```

```
    n |  a_n (count)
----------------------
    1 |            2
    2 |            4
    3 |            7
    4 |           12
    5 |           21
    6 |           37
    7 |           65
    8 |          114
    9 |          200
   10 |          351
```

1. (i) Plot the 3d surfaces and combine them x 3 + y −4 + z 2 = 2 and x 2 − y 2 = 2z

and combine them.

In [34]:
```
var('x y z')
p1 = implicit_plot3d(x^3 + y^(-4) + z^2 - 2, (x,-2,2),(y,0.5,2),(z,-2,2),
                     color='blue', opacity=0.6)
p2 = implicit_plot3d(x^2 - y^2 - 2*z, (x,-2,2),(y,-2,2),(z,-2,2),
                     color='red', opacity=0.6)
show(p1 + p2, axes=True)
```



(ii) Find the surface of revolution of y = cos(x x 2+1) rotated through Z axis by an angle

360 degrees.

In [35]:
```
var('r theta')
f = cos(r / (r^2 + 1))

surf = parametric_plot3d(
    (r*cos(theta), r*sin(theta), f),
    (r, -3, 3), (theta, 0, 2*pi),
    color='green', opacity=0.8
)
show(surf, axes=True)
```

1. (i) Find all the partial derivatives up to order 2 for the function f(x, y) = sin−1 (x 3−y 3 x 2+y2)

In [36]:
```
var('x y')
f = arcsin((x^3 - y^3) / (x^2 + y^2)^(Rational(1,2)))

fx  = diff(f, x)
fy  = diff(f, y)
fxx = diff(f, x, 2)
fyy = diff(f, y, 2)
fxy = diff(fx, y)

print("f_x  =", fx.simplify_full())
print("f_y  =", fy.simplify_full())
print("f_xx =", fxx.simplify_full())
print("f_yy =", fyy.simplify_full())
print("f_xy =", fxy.simplify_full())
```

```
f_x  = (x^4 + 3*x^2*y^2 + 2*x*y^3)/((x^4 + 2*x^2*y^2 + y^4)*sqrt(-(x^6 - 2
*x^3*y^3 + y^6 - x^4 - 2*x^2*y^2 - y^4)/(x^4 + 2*x^2*y^2 + y^4)))
f_y  = -(2*x^3*y + 3*x^2*y^2 + y^4)/((x^4 + 2*x^2*y^2 + y^4)*sqrt(-(x^6 -
2*x^3*y^3 + y^6 - x^4 - 2*x^2*y^2 - y^4)/(x^4 + 2*x^2*y^2 + y^4)))
f_xx = (x^11 - 6*x*y^10 - 2*y^11 + 2*(x^2 + 1)*y^9 - 10*x^4*y^5 - 6*(x^3 -
x)*y^8 + (3*x^4 - 2*x^2)*y^7 - 2*(6*x^5 - 5*x^3)*y^6 + (3*x^7 + 2*x^5)*y^4
+ 3*(3*x^8 - 2*x^6)*y^3 + 2*(4*x^9 - x^7)*y^2)/((x^10 + 5*x^8*y^2 + 10*x^6
*y^4 + 10*x^4*y^6 + 5*x^2*y^8 + y^10)*(-(x^6 - 2*x^3*y^3 + y^6 - x^4 - 2*x
^2*y^2 - y^4)/(x^4 + 2*x^2*y^2 + y^4))^(3/2))
f_yy = (2*x^11 - 9*x^3*y^8 - 8*x^2*y^9 - y^11 - 2*x^9 + 6*x^3*y^6 - (3*x^4
- 2*x^2)*y^7 + 2*(6*x^6 - x^4)*y^5 - (3*x^7 - 10*x^5)*y^4 + 2*(3*x^8 - 5*x
^6)*y^3 - 2*(x^9 - x^7)*y^2 + 6*(x^10 - x^8)*y)/((x^10 + 5*x^8*y^2 + 10*x^
6*y^4 + 10*x^4*y^6 + 5*x^2*y^8 + y^10)*(-(x^6 - 2*x^3*y^3 + y^6 - x^4 - 2*
x^2*y^2 - y^4)/(x^4 + 2*x^2*y^2 + y^4))^(3/2))
f_xy = (9*x^2*y^9 + 4*x*y^10 - 2*x^6*y^3 - 2*x*y^8 + 6*(x^4 - x^2)*y^7 -
(9*x^5 - 2*x^3)*y^6 + (9*x^6 - 10*x^4)*y^5 - 2*(3*x^7 - 5*x^5)*y^4 - 3*(3*
x^9 - 2*x^7)*y^2 - 2*(2*x^10 - x^8)*y)/((x^10 + 5*x^8*y^2 + 10*x^6*y^4 + 1
0*x^4*y^6 + 5*x^2*y^8 + y^10)*(-(x^6 - 2*x^3*y^3 + y^6 - x^4 - 2*x^2*y^2 -
y^4)/(x^4 + 2*x^2*y^2 + y^4))^(3/2))
```

(ii) Find the maxima and minima of xy + 64 x + 64 y

In [37]:
```
var('x y')
f = x*y + 64/x + 64/y

fx = diff(f, x)
fy = diff(f, y)

critical = solve([fx == 0, fy == 0], x, y, solution_dict=True)
print("Critical points:", critical)

fxx = diff(f, x, 2)
fyy = diff(f, y, 2)
fxy = diff(fx, y)

for pt in critical:
    xv, yv = pt[x], pt[y]
    D = (fxx*fyy - fxy^2).subs(pt)
    fxx_val = fxx.subs(pt)
    print(f"\nAt ({xv}, {yv}):")
    print(f"  D = {D}")
    if D > 0 and fxx_val > 0:
        print("  → Local MINIMUM, f =", f.subs(pt))
    elif D > 0 and fxx_val < 0:
        print("  → Local MAXIMUM, f =", f.subs(pt))
    else:
        print("  → Saddle point or inconclusive")
```

```
Critical points: [{x: 4, y: 4}, {x: -2*I*sqrt(3) - 2, y: -2*I*sqrt(3) -
2}, {x: 2*I*sqrt(3) - 2, y: 2*I*sqrt(3) - 2}]

At (4, 4):
  D = 3
  → Local MINIMUM, f = 48

At (-2*I*sqrt(3) - 2, -2*I*sqrt(3) - 2):
  D = 16384/(-2*I*sqrt(3) - 2)^6 - 1
  → Local MINIMUM, f = (-2*I*sqrt(3) - 2)^2 + 128/(-2*I*sqrt(3) - 2)

At (2*I*sqrt(3) - 2, 2*I*sqrt(3) - 2):
  D = 16384/(2*I*sqrt(3) - 2)^6 - 1
  → Local MINIMUM, f = (2*I*sqrt(3) - 2)^2 + 128/(2*I*sqrt(3) - 2)
```